

Architecture Définitive – DAR_Cyber

1. Statut du document

Ce document décrit l'architecture technique complète du projet DAR_Cyber. Il constitue une référence structurante pour l'ensemble du développement, en traduisant les exigences fonctionnelles en choix d'implémentation concrets. Il permet également de justifier les décisions techniques lors de la soutenance.

2. Objectif de l'architecture

L'architecture vise à concevoir un système de supervision réseau capable de collecter des données depuis des agents endpoint, de les traiter en temps réel et différé, puis de produire des alertes exploitables. Elle doit assurer la cohérence des flux de données, la fiabilité du stockage et la lisibilité des résultats pour un analyste SOC.

3. Description technique globale

Le système repose sur une architecture centralisée autour d'un serveur SOC. Ce serveur expose une API FastAPI qui agit comme point d'entrée unique. Les endpoints envoient des données via des requêtes HTTP. Ces données sont validées par des schémas Pydantic, puis persistées dans PostgreSQL. Les traitements lourds (corrélation, détection, génération d'alertes) sont délégués à Celery via Redis, ce qui permet de ne pas bloquer le thread principal de l'API.

4. Backend FastAPI

Le backend est développé avec FastAPI, permettant une gestion performante des routes HTTP asynchrones. Chaque module métier est structuré en routes, services et modèles. SQLAlchemy est utilisé pour l'ORM afin de mapper les objets Python aux tables PostgreSQL. Le backend gère également l'authentification, la validation des données et l'orchestration des traitements.

5. Base de données PostgreSQL

PostgreSQL constitue la source de vérité du système. Il stocke les entités critiques telles que les utilisateurs, les événements, les actifs et les alertes. Les relations entre ces entités permettent d'assurer la traçabilité complète des événements. Les index et contraintes garantissent la performance et l'intégrité des données.

6. Traitement asynchrone (Redis + Celery)

Redis est utilisé comme broker de messages pour Celery. Lorsqu'un événement est reçu, une tâche est envoyée dans la file Redis. Le worker Celery récupère cette tâche et exécute les règles de détection ou de corrélation. Cette architecture permet de gérer des volumes importants tout en maintenant une API réactive.

7. Frontend

Le frontend est une interface web permettant de visualiser les données. Il consomme l'API via des appels REST. Il affiche les dashboards, les alertes, les événements et les exports. L'objectif est d'offrir une interface lisible et exploitable par un analyste.

8. Agent Endpoint

L'agent endpoint est responsable de la collecte de données locales. Il surveille le trafic réseau, analyse les logs et envoie des événements au serveur SOC. Il fonctionne de manière autonome et communique uniquement avec l'API.

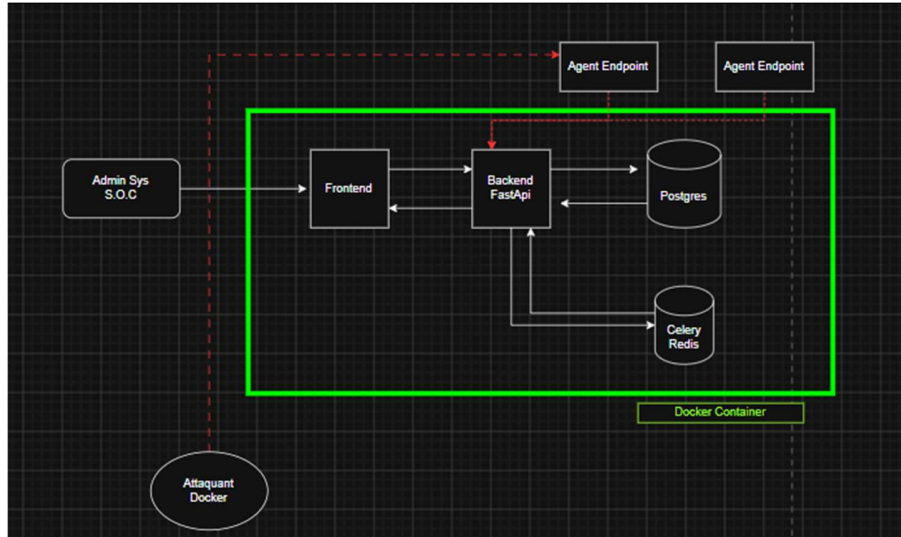
9. Simulateur Attacker

Le conteneur attacker permet de simuler des attaques contrôlées. Il génère des scénarios reproductibles comme des scans de ports ou des connexions suspectes. Ces scénarios permettent de valider le bon fonctionnement du système de détection.

10. Flux technique détaillé

Lorsqu'un événement est généré, il est envoyé à l'API. L'API valide et enregistre les données. Ensuite, une tâche est envoyée à Celery. Le worker traite l'événement, applique les règles et génère éventuellement une alerte. Cette alerte est ensuite disponible via l'interface utilisateur.

11. Schéma d'architecture



12. Conclusion

Cette architecture offre un équilibre entre simplicité et robustesse. Elle est adaptée à un MVP tout en restant extensible. Elle permet de démontrer clairement la chaîne complète de traitement des données, depuis la collecte jusqu'à la production de preuves.